# Vision-Language-Action Integration for Natural-Language Robot Arm Control

(65011539) Sittipon Kumda

(65011499) Rachata Pratheep Na Thalang

(65011525) Sippaphat Pitiwan

(65011563) Tanakorn Youngmeesuk

(65011586) Thanchanok Nuengcham Nong

A THESIS SUBMITTED IN PARTIAL FULFILLMENT

OF THE REQUIREMENT FOR THE DEGREE OF

BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING

SCHOOL OF ENGINEERING

KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG

2026

KMITL-2026-EN-X-XXX-XXX

# Vision-Language-Action Integration for Natural-Language Robot Arm Control

(65011539) Sittipon Kumda

(65011499) Rachata Pratheep Na Thalang

(65011525) Sippaphat Pitiwan

(65011563) Tanakorn Youngmeesuk

(65011586) Thanchanok Nuengcham Nong

A THESIS SUBMITTED IN PARTIAL FULFILLMENT

OF THE REQUIREMENT FOR THE DEGREE OF

BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING

SCHOOL OF ENGINEERING

KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG

2026

KMITL-2026-EN-X-XXX-XXX

| | |
|---|---|
| **Thesis** | Vision-Language-Action Integration for Natural-Language Robot Arm Control with Improved Generalization to Unseen Manipulation Tasks |
| **Students** | Mr. Sittipon Kumda<br>Mr. Rachata Pratheep Na Thalang<br>Mr. Sippaphat Pitiwan<br>Mr. Tanakorn Youngmeesuk<br>Ms. Thanchanok Nuengcham Nong |
| **Student IDs** | 65011539, 65011499, 65011525, 65011563, 65011586 |
| **Degree** | Bachelor of Engineering |
| **Program** | Robotics and AI Engineering |
| **Year** | 2026 |
| **Thesis Advisor** | Prof. Dr. Poom Konghuayrob |


# ABSTRACT IN ENGLISH


This thesis presents a Vision-Language-Action (VLA) framework for robot arm manipulation in which visual perception, language understanding, and action generation are integrated into a single control pipeline. The work addresses key limitations of current VLA practice in low-resource settings: high data collection cost, limited access to high-memory hardware, and fragmented integration between multimodal AI and robot-controller execution.

The proposed architecture combines three coordinated components: a Vision-Language Model (VLM) core for instruction grounding, a Vision Expert for task-relevant visual feature extraction, and an Action Expert for executable robot action generation. The system receives an RGB scene observation and a free-form natural-language command, then produces structured manipulation actions suitable for safe robot-controller execution through a TCP/IP communication adapter. The design specifically targets a commercial setup with limited dataset volume and an 8 GB VRAM class training budget.

Experiments on representative robot manipulation tasks show that explicit VLM + Vision Expert + Action Expert integration improves action validity and task success compared with partial baselines. Evaluation across simulation and real-robot domains, including zero-shot settings, shows that the system follows natural-language instructions reliably, maintains practical real-time latency, and retains competitive performance under constrained training resources. The results support the feasibility of practical low-resource VLA-based robot control under realistic computation, data, and integration constraints.



**Keywords:** Vision-Language-Action, Robot Manipulation, Natural-Language Robot Control, Multimodal Integration, Generalization, Embodied AI

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Prof. Dr. Poom Konghuayrob, my thesis advisor, for his patient guidance, constructive criticism, and consistent encouragement throughout every stage of this research. His deep knowledge of robotics and AI systems provided indispensable direction whenever this work encountered technical obstacles.

I am grateful to the faculty of the Robotics and AI Engineering programme at King Mongkut's Institute of Technology Ladkrabang for providing the academic foundation, the computational resources, and the laboratory environment that made this project possible.

I also wish to thank my fellow students and research colleagues for their valuable discussions, their willingness to review drafts, and their practical help during hardware assembly and testing.

Finally, I owe a profound debt of gratitude to my family for their unwavering support, patience, and encouragement throughout my undergraduate studies. This work is dedicated to them.

(65011539) Sittipon Kumda

(65011499) Rachata Pratheep Na Thalang

(65011525) Sippaphat Pitiwan

(65011563) Tanakorn Youngmeesuk

(65011586) Thanchanok Nuengcham Nong

# TABLE OF CONTENTS

Page

# LIST OF TABLES

**Table** **Page**

# LIST OF FIGURES

**Figure** **Page**

# LIST OF ABBREVIATIONS AND SYMBOLS

| | |
|---|---|
| C | Specific heat |
| P | Pressure |
| V | Volume |

# CHAPTER 1

# INTRODUCTION

## 1.1 Background and Motivation

Vision-Language Models (VLMs) have recently demonstrated high global impact in multimodal reasoning, instruction following, and human-computer interaction. They can understand free-form text and visual context in a way that significantly reduces the interface barrier between human intent and machine behavior. A natural next step is to migrate this intelligence from digital environments to the physical world, where robots must perceive, reason, and act under real constraints.

However, building practical Vision-Language-Action (VLA) systems remains difficult. Most high-performing solutions require large amounts of specialized robot datasets, long data-collection time, and heavy hardware setups. These requirements make VLA development inaccessible for many university teams and small labs. In particular, real-world robot data collection is expensive, and large GPU memory is not always available for end-to-end training and deployment.

This thesis is motivated by the following question: can an end-to-end pipeline enable VLA development under a minimal commercial setup, including low-volume data and moderate GPU resources? To answer this, we design and evaluate a practical VLA framework that combines multimodal understanding and robot control in one trainable pipeline while maintaining feasibility for constrained environments.

## 1.2 Problem Identification

Based on the motivation above, the core problem is not only model quality but development accessibility. Existing VLA practices are often constrained by high data requirements and expensive compute, which slows down experimentation and limits reproducibility across institutions.

**Gap 1: High Entry Cost for VLA Development.** Many published VLA pipelines depend on long-duration robot data collection (frequently more than 10 hours per setup), large and diverse task sets, and powerful multi-GPU infrastructure. This is misaligned with low-cost academic or commercial prototyping environments.

**Gap 2: Insufficient Low-Resource Design Principles.** There is limited guidance on how to design a VLA pipeline that still works when data volume is low, task diversity is narrow, and hardware is constrained (e.g., 8 GB VRAM).

**Gap 3: Fragmented AI-to-Robot Integration.** Even when multimodal models perform well, practical deployment fails if communication between AI outputs and robot controller commands is not standardized, safe, and composable.

**Gap 4: Incomplete Sim-to-Real Validation.** Evaluation is often focused on simulation only, while real-robot performance, zero-shot transfer, and deployment safety constraints are underreported.

Building on these identified gaps, this thesis poses the following primary research question:

*How can a practical end-to-end Vision-Language-Action pipeline be designed to train and deploy natural-language robot manipulation under low-data and limited-GPU conditions while preserving usable performance in both simulation and real-world settings?*

The following secondary questions are also addressed:

1) How should an end-to-end VLA pipeline be structured so that model outputs are directly usable for robot execution through a stable AI-to-robot interface?

2) Which design choices are most effective when dataset duration is around one hour and training must fit within an 8 GB VRAM budget?

3) To what extent can the resulting system follow natural-language commands and generalize under zero-shot conditions in simulation and real-world tests?

## 1.3 Objectives

The overarching objective of this research is to make VLA development feasible under commercially accessible resources. The specific objectives are as follows:

1) To develop an end-to-end Vision-Language-Action pipeline that can be trained and deployed with low dataset volume and limited GPU resources.

2) To build a VLA model that can steer a robot arm according to natural-language commands with reliable executable outputs.

## 1.4 Key Knowledge Resources

This research requires interdisciplinary knowledge across robotics, mathematics, machine learning, and systems engineering. The main knowledge resources are:

1) **Robot Kinematics**: forward/inverse kinematics, workspace limits, and joint-space to task-space mapping for manipulator control.

2) **Mathematics**: calculus, probability, and linear algebra for model optimization, uncertainty reasoning, and representation learning.

3) **Data Science and Data Analysis**: dataset construction, data quality auditing, distribution analysis, and experiment interpretation.

4) **Modern Machine Learning**: representation learning, sequence modeling, multimodal alignment, and training stabilization strategies.

5) **Networking and Protocols**: TCP/IP communication for robust, low-latency AI-to-robot command transport and runtime synchronization.

## 1.5 Scope and Limitations

The scope of this research is delimited as follows. The target platform is a Dobot CR5 robot arm for tabletop manipulation with RGB observations and natural-language instructions. The dataset scope is intentionally constrained to a low-resource regime (approximately one hour of robot motion and three task types), and training/deployment is designed to fit commercial GPU hardware around 8 GB VRAM. The study covers both sim-

ulation and real-world evaluation, including zero-shot setups. Full autonomous navigation, large-scale multi-robot coordination, and formal safety certification are outside this scope.

## 1.6 Contributions

The principal contributions of this thesis are as follows:

1) **A low-resource end-to-end VLA pipeline** for training and deployment under limited dataset and GPU conditions.

2) **A practical AI-to-robot communication interface** that converts model outputs into composable machine commands over TCP/IP for Dobot control.

3) **A natural-language control capability** that enables robot steering from user commands without manual per-task scripting.

4) **A sim-and-real evaluation framework** covering success rate and zero-shot transfer in two domains.

5) **An implementation playbook** spanning dataset collection, model development, training objective design, inference analysis, and model improvement.

## 1.7 Thesis Organisation

The remainder of this thesis is structured as follows. Chapter 2 reviews related theories and prior work on multimodal robot learning, natural-language instruction following, and action modeling for manipulation. Chapter 3 presents the proposed VLA system design, including module integration (VLM + Vision Expert + Action Expert), design constraints, and implementation strategy. Chapter 4 reports experimental results on manipulation tasks, including instruction-following performance, generalization to unseen tasks, and real-time latency analysis. Chapter 5 summarizes key findings, limitations, and future directions for general-purpose intelligent robot systems.

# CHAPTER 2

# RELATED THEORIES AND LITERATURE REVIEW

This chapter synthesizes the theoretical and empirical foundations that motivate the proposed Vision-Language-Action (VLA) framework. Rather than surveying prior work in isolation, the chapter connects core theories to the concrete problem addressed in this thesis: building practical natural-language robot manipulation under limited data and constrained compute.

The discussion is organized into five themes: (1) limitations of classical robot pipelines, (2) foundations of Vision-Language Models (VLMs), (3) action modeling for manipulation, (4) integrated VLA system design in recent literature, and (5) generalization and transfer challenges. These themes form the conceptual bridge from problem statement (Chapter 1) to methodology (Chapter 3).

## 2.1 LIMITATIONS OF CLASSICAL ROBOT SYSTEMS

Classical robot systems are typically composed of hand-engineered perception, planning, and control modules. This design provides predictable behavior in fixed conditions, but it is often task-specific and difficult to scale to open-world settings. Changes in objects, layouts, or command style can require redesign of features, rules, and control logic.

A major weakness is limited generalization. Policies tuned for narrow task templates frequently fail in unseen scenarios, especially when visual context and language instructions vary simultaneously. In practical terms, this leads to high maintenance cost, brittle behavior outside curated test distributions, and poor accessibility for non-expert users.

Another structural limitation is interface rigidity. Classical systems commonly depend on deterministic command schemas or finite-state interfaces that assume predefined task structure. Such assumptions conflict with natural-language interaction, where intent is often underspecified, rephrased, or context dependent. This gap motivates multimodal approaches that can jointly interpret language and visual context before action generation.

## 2.2 VISION-LANGUAGE MODEL FOUNDATIONS

### 2.2.1 Transformer Backbone

Modern VLMs are built on the Transformer architecture [12], which models long-range dependencies using multi-head self-attention. Given query, key, and value matrices, the core operation is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2.1}$$

This mechanism supports flexible context integration across image and language tokens.

For embodied decision-making, this property is particularly relevant because robot policies must condition on long contextual chains: instruction text, scene structure, task history, and safety constraints. Attention-

based architectures provide a practical mechanism for such cross-token dependency modeling, especially when perception and language are encoded in a shared sequence space.

### 2.2.2 Vision-Language Instruction Following

Recent VLMs demonstrate strong multimodal instruction-following behavior through large-scale pretraining and instruction tuning [10, 9]. For robotic systems, this capability enables grounding free-form user requests in scene context, reducing the need for rigid command interfaces.

However, VLM output must still be translated into robot-executable actions with low latency and high reliability, which motivates explicit action modeling beyond language generation alone.

From a systems perspective, two observations are important. First, language-level correctness does not imply control-level correctness; syntactically coherent text may still map to infeasible motion. Second, VLM confidence is not equivalent to execution safety. Therefore, practical robotic deployment requires an intermediate control layer that validates and structures model outputs before hardware dispatch.

## 2.3 ACTION MODELING FOR ROBOT MANIPULATION

Action generation for manipulation usually requires a design choice between discrete and continuous representations.

- **Discrete actions** are easier to validate and parse but may reduce motion fidelity.
- **Continuous actions** allow fine-grained control but are more sensitive to model errors and deployment noise.

In practice, many systems use structured or mixed forms to balance robustness and expressiveness. For real-time control, action validity and execution safety are as important as semantic correctness.

In low-resource settings, this representation choice becomes more critical. With limited demonstration diversity, purely continuous policies may overfit trajectory statistics and degrade sharply in unseen configurations. Hybrid action schemas, where high-level decisions are discretized and low-level motion remains continuous, can improve controllability while preserving enough dexterity for manipulation tasks.

Action modeling in this thesis therefore emphasizes three properties:

1) **Executability**: model outputs must map directly to controller-valid commands.
2) **Safety compatibility**: outputs must satisfy workspace and joint-range constraints before execution.
3) **Latency feasibility**: decoding and validation must remain practical for interactive command cycles.

## 2.4 VISION-LANGUAGE-ACTION INTEGRATION

### 2.4.1 VLA Systems in Prior Work

Research on Vision-Language-Action integration has shown that combining multimodal reasoning with action prediction can improve robotic adaptability. RT-2 [2] demonstrated language-conditioned action

generation, PaLM-E [4] highlighted embodied multimodal context integration, and OpenVLA [8] provided an open framework for scalable VLA training and evaluation.

These systems indicate that end-to-end multimodal-action learning can outperform strictly task-specific pipelines. At the same time, deployment still faces constraints in compute, latency, data scale, and module coordination.

An important methodological implication is that integration quality is not solely a model-size problem. Even strong backbones can fail if perception features, language intent representations, and action outputs are weakly coupled. Consequently, architectural clarity and interface design are central to robust VLA behavior.

### 2.4.2 Need for Modular Integration

A practical VLA system must synchronize three capabilities:

1) language grounding and task understanding,
2) visual feature extraction under scene variation,
3) robust generation of executable manipulation actions.

This motivates the modular design used in this thesis: VLM core, Vision Expert, and Action Expert connected by explicit intermediate representations.

The modular strategy is adopted not as a rejection of end-to-end learning, but as a practical compromise between trainability and interpretability. It enables targeted diagnosis of failure sources (language grounding versus visual ambiguity versus action decoding) while still supporting end-to-end optimization during later refinement stages.

## 2.5 GENERALIZATION IN ROBOT LEARNING

Generalization remains a central challenge in embodied AI. Training performance on seen tasks does not guarantee transfer to unseen objects, compositions, and instruction phrasings. Dataset limitations amplify this gap because robot demonstrations are expensive to collect at scale.

Current literature suggests several factors that improve transfer:

- broader task diversity during training,
- stronger language variation in instruction data,
- robust visual representations across domain shifts,
- stable action interfaces that reduce execution ambiguity.

These findings directly inform the design constraints and evaluation protocol of this thesis, where unseen-task performance is treated as a primary metric rather than a secondary result.

For low-resource VLA development, generalization should be interpreted as a ratio, not only an absolute score. That is, the key question is how much capability is retained when moving from seen to unseen conditions under fixed resource budgets. This view aligns with the thesis objective of practical deployment rather than leaderboard optimization.

## 2.6 MECHANISTIC INTERPRETABILITY OF VISION-LANGUAGE MODELS

A complementary research direction to standard VLA training is mechanistic interpretability: understanding *how* VLMs encode and process information internally, rather than only measuring what they output. This perspective is increasingly relevant for robotics, where emergent capabilities such as spatial reasoning or action planning may already be latent within pretrained VLMs and could be surfaced without additional supervised training.

### 2.6.1 Feature Superposition and Sparse Decomposition

Neural networks, particularly large Transformer-based models, are believed to store far more concepts than they have neurons through a phenomenon called *superposition* [5]. Individual neurons rarely correspond to cleanly interpretable concepts. Instead, each neuron participates in representing many overlapping features simultaneously, making it difficult to isolate specific behavioral directions by direct inspection.

Sparse decomposition addresses this by seeking a set of *basis directions* in activation space such that any forward-pass activation can be expressed as a sparse linear combination of those directions. Each basis direction ideally corresponds to a single interpretable feature. Sparse Autoencoders (SAEs) provide a tractable mechanism for learning such decompositions from data.

### 2.6.2 Sparse Autoencoders for Feature Discovery

A Sparse Autoencoder is a two-layer bottleneck network trained to reconstruct model activations through a high-dimensional intermediate representation under a sparsity constraint [3]. Given an input activation vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$, an SAE produces:

$$\hat{\mathbf{x}} = W_{\text{dec}}\,\text{TopK}(W_{\text{enc}}\mathbf{x} + \mathbf{b}_{\text{enc}}) + \mathbf{b}_{\text{dec}}, \tag{2.2}$$

where $W_{\text{enc}} \in \mathbb{R}^{d_{\text{sae}} \times d_{\text{model}}}$ and $W_{\text{dec}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{sae}}}$ are the encoder and decoder weight matrices, and $\text{TopK}(\cdot)$ retains only the $k$ largest non-negative pre-activations, zeroing the rest. The expansion ratio $d_{\text{sae}}/d_{\text{model}}$ controls the dictionary size relative to the original representation space. The TopK constraint directly enforces sparsity rather than relying on an $L_1$ penalty, which produces more stable and monosemantic features in practice [6].

Once trained, each column $\mathbf{d}_i$ of the decoder matrix $W_{\text{dec}}$ defines a *feature direction* in the model's hidden space. A high activation value for feature $i$ on a given input implies that the model's internal state strongly aligns with that direction during processing of that input.

### 2.6.3 Activation Steering via Feature Injection

Activation steering is a technique for directly modifying model behavior by adding a target direction to its internal activations during inference [13]. If a feature direction $\mathbf{d}_i \in \mathbb{R}^{d_{\text{model}}}$ has been identified as encoding a concept of interest, the steered output at layer $\ell$ is:

$$\tilde{\mathbf{h}}_t^{(\ell)} = \mathbf{h}_t^{(\ell)} + \alpha\,\mathbf{d}_i, \tag{2.3}$$

where $\mathbf{h}_t^{(\ell)}$ is the residual-stream hidden state at token position $t$ and layer $\ell$, and $\alpha$ is an injection strength hyperparameter. This technique bypasses the need for gradient updates and allows targeted

behavioral modification at inference time.

For robotic applications, activation steering is particularly relevant because it provides a lightweight path to zero-shot specialization: a general-purpose VLM can potentially be steered toward manipulation-relevant output modes without any task-specific fine-tuning, data collection, or retraining cost.

### 2.6.4 Application to Robotic Trajectory Planning

Prior work has demonstrated that language models encode factual, logical, and stylistic concepts as recoverable feature directions. The hypothesis motivating the present work is that a multimodal VLM trained at scale may also encode latent representations of *spatial kinematics and manipulation procedures* that are not surfaced by standard prompting, but that can be discovered via SAE analysis and activated via steering. This motivates the VLM mechanistic interpretability sub-project described in Chapter 3, which applies SAE-based feature discovery and activation steering to Qwen3-VL-2B for zero-shot robotic trajectory planning.

## 2.7 CHAPTER SUMMARY

This chapter established the theoretical and empirical basis for the proposed Vision-Language-Action approach. Prior work confirms the potential of multimodal robot learning but also highlights unresolved issues in flexibility, natural-language interaction, and generalization. These gaps motivate the VLA architecture developed in Chapter 3, where a VLM, Vision Expert, and Action Expert are integrated for practical robot manipulation under real-world constraints. Additionally, the chapter surveyed mechanistic interpretability techniques—in particular Sparse Autoencoders and activation steering—which form the theoretical basis for the complementary VLM interpretability sub-project described in the same chapter.

# CHAPTER 3

# RESEARCH METHODOLOGY AND SYSTEM DESIGN

This chapter presents the methodology and system design of the proposed Vision-Language-Action (VLA) framework for robot manipulation under constrained resources. The chapter focuses on three project priorities: (1) constructing an end-to-end VLA pipeline that is practical for commercial setups, (2) satisfying data, hardware, and safety constraints for Dobot CR5 deployment, and (3) defining a clear evaluation protocol across simulation and real-world domains.

To improve traceability between design and evidence, the chapter is structured from requirements to implementation and then to evaluation. Each design decision is explicitly linked to one or more project constraints introduced in Chapter 1.

## 3.1 METHODOLOGY OVERVIEW

The research workflow is organized as an implementation pipeline from communication infrastructure to model refinement:

1) **AI-to-Robot Communication Interface**: build a module that converts Dobot TCP/IP command primitives into a machine-learning-friendly composable action format and vice versa.

2) **Dataset Collection**: construct training data from two sources: (a) simulation trajectories in Isaac Lab exported to LeRobot format, and (b) real-world human demonstrations on the physical robot.

3) **Model Development**: implement and integrate multimodal perception, language grounding, and action prediction modules into one end-to-end model.

4) **Training Objective Design**: define losses and optimization settings for stable learning under low-data and limited-memory conditions.

5) **Inference and Model Analysis**: inspect prediction behavior, latency, failure modes, and sim-to-real transfer quality.

6) **Model Improvement**: apply targeted refinement based on analysis, including data balancing, prompt/interface tuning, and architecture adjustments.

At a high level, the learning problem is to estimate a policy $\pi_\theta$ that maps multimodal context to controller-executable actions:

$$a_t = \pi_\theta(I_t, \mathcal{U}, h_t),$$

where $I_t$ is the RGB observation, $\mathcal{U}$ is the natural-language instruction, and $h_t$ is optional history/state context. The methodology seeks to maximize task success under three constraints: limited demonstrations, limited memory, and safety-constrained execution.

## 3.2 PROBLEM-DRIVEN DESIGN REQUIREMENTS

The system design is directly derived from the project constraints. Unlike large-scale VLA studies that rely on broad datasets and high-end servers, this thesis targets a minimal commercial setup. Therefore, design

decisions are driven by three major constraints.

### 3.2.1 Dataset Quantity and Variety Constraint

The available dataset is intentionally limited: approximately one hour of total robot motion with only three task families. This is significantly smaller than many VLA pipelines that use more than ten hours of demonstrations and broader task diversity. As a result, the methodology prioritizes sample efficiency, high data quality, cross-domain consistency (sim and real), and curriculum-style task coverage.

In practical terms, this means the pipeline avoids data-hungry assumptions and instead prioritizes representation reuse, careful trajectory normalization, and targeted coverage of failure-prone scenarios.

### 3.2.2 Hardware Constraint (Commercial GPU)

The model architecture must fit within practical hardware specifications, specifically around 8 GB of VRAM. This restricts parameter count, sequence length, and batch-size strategy, and motivates memory-aware training techniques such as mixed precision, gradient accumulation, compact tokenization, and lightweight heads.

This constraint also affects experiment planning: ablations and evaluation schedules must be designed to preserve comparability without requiring large parallel runs.

### 3.2.3 Safety and Inference Standard Constraint

For safe and repeatable operation on Dobot CR5, robot joint-position standards are fixed before inference to define a consistent workspace and startup condition. The exact joint preset is recorded in the experiment configuration file and should be reported as `[Joint preset values for Dobot CR5 to be specified in final version]`. This standardization reduces run-to-run variance and lowers risk during autonomous execution.

The safety layer is treated as a first-class system component rather than a post hoc filter. This design choice is critical when model predictions are generated from natural-language input, where ambiguity can propagate to unsafe control proposals.

## 3.3 VISION-LANGUAGE-ACTION SYSTEM ARCHITECTURE

### 3.3.1 Input and Output Definition

At each control cycle, the system receives:

- an RGB observation $I_t$, and
- a natural-language instruction $\mathcal{U}$ from the user.

The system outputs a structured action command executable by the robot arm controller. The output is designed to be composable with the TCP/IP communication interface so it can be translated safely into Dobot command streams.

Operationally, the architecture separates semantic grounding from command realization. This improves debuggability and allows independent validation of perception-language alignment and action executability.

### 3.3.2 VLM Core

The VLM module performs multimodal understanding and instruction grounding. It extracts intent from natural language, associates intent with current scene context, and generates a high-level task state for downstream action production.

The VLM output is intentionally constrained to a structured intermediate task state instead of free-form control text. This reduces ambiguity at the interface with the action generator.

### 3.3.3 Vision Expert

The Vision Expert provides task-relevant visual features (object cues, target relations, workspace context, and target hints) for robust action planning. It is designed to improve perception reliability under object and scene variation.

To support low-resource training, the Vision Expert emphasizes transferable features that remain stable across simulation and real camera conditions.

### 3.3.4 Action Expert

The Action Expert converts grounded task representations into executable robot manipulation commands. The module is constrained by communication safety rules, workspace limits, and robot kinematics, ensuring that generated actions remain valid for runtime execution.

The Action Expert also acts as a control regularizer: it converts semantic intent into bounded command trajectories that are compatible with controller timing and packet format requirements.

### 3.3.5 Module Coordination

The three modules are connected through explicit intermediate representations:

a) **Language-grounded task state** from VLM,

b) **Task-conditioned visual features** from Vision Expert,

c) **Executable action sequence** from Action Expert.

This separation improves maintainability and enables targeted optimization of each module while keeping end-to-end behavior coherent.

In addition, the architecture introduces a protocol adapter between the Action Expert and Dobot controller. This adapter maps model-level action semantics to TCP/IP command payloads with explicit validation rules (range checks, rate checks, and command structure checks) before dispatch.

The protocol adapter provides deterministic verification before hardware dispatch, which is essential for reproducibility and for safe execution in real-robot tests.

## 3.4 IMPLEMENTATION STRATEGY

### 3.4.1 Dataset Collection Strategy

The data pipeline combines simulation and real-world trajectories:

1) Generate simulation trajectories in Isaac Lab.

2) Export trajectories to LeRobot-compatible format.

3) Collect human demonstrations on the physical Dobot CR5.

4) Normalize both sources into a unified schema of observation, instruction, and action tuples.

This two-source design improves sample efficiency: simulation contributes scalable coverage of trajectory patterns, while real demonstrations provide embodiment-specific corrections for dynamics and sensing mismatch.

### 3.4.2 Model Development and Training Strategy

The system is trained and tuned in stages:

1) Initialize the VLA backbone with pretrained multimodal knowledge.
2) Adapt perception and action heads to robot-manipulation dynamics.
3) Train with low-resource-aware objectives for stability and action validity.
4) Perform end-to-end tuning to reduce module mismatch and command noise.

The training objective combines command prediction and execution validity terms:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{action}} + \lambda_2 \mathcal{L}_{\text{validity}} + \lambda_3 \mathcal{L}_{\text{alignment}},$$

where $\mathcal{L}_{\text{action}}$ supervises target actions, $\mathcal{L}_{\text{validity}}$ penalizes unsafe or non-executable outputs, and $\mathcal{L}_{\text{alignment}}$ preserves language-vision-task consistency.

### 3.4.3 Inference and Model Analysis

After training, inference behavior is analyzed in both simulation and physical setups, including latency profile, command validity, failure taxonomy, and transfer behavior. This analysis identifies whether errors are caused by perception ambiguity, language misgrounding, or action decoding instability.

Failure analysis is organized by stage so corrective interventions can be applied with minimal retraining overhead.

### 3.4.4 Model Improvement Loop

Based on analysis findings, iterative improvements are applied through data balancing, task reweighting, communication-level safeguards, and architectural tuning.

This closed-loop process is central to low-resource development because each improvement cycle must produce measurable gains without expensive data re-collection.

### 3.4.5 Latency and Memory Optimization

To satisfy real-time robot-control requirements, the implementation applies practical optimization techniques, including mixed precision inference, efficient token/image preprocessing, and lightweight module interfaces that remain feasible on 8 GB VRAM.

Together, these optimizations convert the architecture from a proof-of-concept model into an executable pipeline for interactive robot operation.

## 3.5 EVALUATION PROTOCOL FOR MANIPULATION TASKS

### 3.5.1 Evaluation Objectives

Evaluation is designed to measure whether the system satisfies the project goals:

- follows natural-language instructions,
- generalizes to unseen tasks,
- maintains stable manipulation output,
- satisfies real-time execution constraints.

The protocol deliberately separates robustness assessment (success-rate domain) from transfer assessment (zero-shot domain), because these dimensions often improve at different rates in constrained-data training.

### 3.5.2 Evaluation Domains

The protocol consists of two main domains:

1) **Success-Rate Evaluation**: assess task completion under standard conditions.
2) **Zero-Shot Evaluation**: assess transfer to unseen combinations without additional fine-tuning.

Each domain is evaluated in both environments:

- On simulation,
- On real robot.

This matrix of {success-rate, zero-shot} × {sim, real} is used to distinguish pure policy quality from embodiment transfer effects.

### 3.5.3 Task Categories

The benchmark includes representative robot manipulation tasks such as:

1) pick-and-place,
2) stacking,
3) drawer or container interaction,
4) multi-step object sequencing.

### 3.5.4 Evaluation Metrics

The following metrics are used:

- **Task success rate**: whether generated actions satisfy the command objective in each task.
- **Zero-shot success rate**: performance on unseen combinations relative to standard seen conditions.
- **Action validity**: percentage of outputs that are executable by the robot controller.
- **Latency**: end-to-end action generation time per command cycle.

For comparative reporting, unseen-task retention is additionally summarized as:

$$\text{Generalization Ratio} = \frac{\text{Unseen Task Success}}{\text{Seen Task Success}} \times 100.$$

## 3.6 VLM MECHANISTIC INTERPRETABILITY SUB-PROJECT

This sub-project applies Sparse Autoencoders (SAEs) to the VLM core to discover which internal feature directions drive robotic action generation, and to steer model behavior by injecting those directions at inference time without retraining.

### 3.6.1 Sub-Project Objectives

1) Construct a 10 000-record robot conversation dataset in `call_robot()` format for single-turn and multi-turn task planning.

2) Harvest layer-20 hidden-state activations from 100 000 multimodal robot-task samples to form the SAE training corpus.

3) Train a TopK SAE on the corpus to extract monosemantic feature directions from the VLM residual stream.

4) Identify which feature directions govern robotic action generation and demonstrate that injecting them reliably elicits structured `call_robot()` outputs without any model retraining.

### 3.6.2 System Architecture

**VLM Core.** Qwen3-VL-2B-Thinking [11] is used as the backbone in `bfloat16` precision. A `register_forward_hook` on transformer layer 20 captures the residual-stream hidden state $\mathbf{h}_t^{(20)} \in \mathbb{R}^{2048}$ at each forward pass. Layer 20 (71% depth in a 28-layer model) is chosen because mid-to-deep layers encode the richest mixture of semantic and procedural concepts [3, 6], while layers near the output are too task-specific to serve as general steering directions.

**SAE Module.** A TopK Autoencoder with 16× expansion ($d_{\text{sae}} = 32\,768$, $d_{\text{model}} = 2\,048$) is trained to reconstruct layer-20 activations:

$$\hat{\mathbf{h}} = W_{\text{dec}}\ \text{TopK}_{32}(W_{\text{enc}}\,\mathbf{h} + \mathbf{b}_{\text{enc}}) + \mathbf{b}_{\text{dec}}. \tag{3.1}$$

Only the 32 largest pre-activations are kept per token ($k = 32$), enforcing sparsity without an $L_1$ penalty. Each decoder column $\mathbf{d}_i$ is a learned feature direction that doubles as a steering vector.

**Steering Hook.** The L2-normalized decoder column for a chosen feature is added to every token's hidden state at layer 20:

$$\tilde{\mathbf{h}}_t^{(20)} = \mathbf{h}_t^{(20)} + \alpha\,\frac{\mathbf{d}_i}{\|\mathbf{d}_i\|_2}, \tag{3.2}$$

where $\alpha$ is an operator-controlled injection strength. No weight updates are required.

**Operator Interface.** A Gradio web app provides an image-upload widget and a real-time $\alpha$ slider (range 0–300), allowing operators to audit and adjust steering behavior live.

### 3.6.3 Dataset Construction

A multi-threaded pipeline (10 workers, 30 RPM rate cap) submits scene frames from EO-Data1.5M [7] (`qa-task_planning` split) to Gemini (gemma-3-27b-it), producing 10 000 JSONL records. Each record contains `image_path`, `task_summary`, `is_multi_turn`, and a `conversation` array where every

assistant turn is a `call_robot()` tool call. Multi-turn records follow a three-step grasp sequence (approach $\longrightarrow$ verify $\longrightarrow$ place).

### 3.6.4 Activation Harvesting

Qwen3-VL-2B processes 100 000 samples with full input–output pairs (`add_generation_prompt=False`) so harvested activations reflect the model's state while *generating* a robot action, not just reading a prompt. Up to 128 token positions are sampled randomly per sequence to cap memory usage, yielding up to 12.8 million `float16` vectors saved across 100 shards.

### 3.6.5 SAE Training

Three checkpoints are produced from the harvested corpus:

- **`sae_task_model.pt`**: trained on single-turn activations only; targets single-step manipulation commands.
- **`sae_agentic_model.pt`**: trained on the full corpus including multi-turn sequences; LR $5 \times 10^{-4}$, 30 epochs.
- **`sae_topk_production.pt`**: full corpus, Adam, LR $1 \times 10^{-4}$, 20 epochs, decoder weights initialized as $W_{\text{enc}}^{\top}$ for faster convergence; used in all steering experiments.

### 3.6.6 Feature Identification Protocol

After training, a probe input with the tool-call prefix `call_robot(`pick a` is run through the model. The cosine similarity between the last-token hidden state and every decoder column is computed, and the top-10 features by score are ranked as candidates. Each candidate is then validated by scanning 5 000 EO-Data1.5M samples for its top-5 highest-activating prompts [1], providing a human-readable label for what concept the feature encodes. Confirmed candidates are swept across $\alpha \in \{0, 2, 5, 10, 20, 50, 80\}$ using two contrasting prompts in parallel: a general knowledge question and a robot trajectory planning request. A repetition penalty of 1.1 suppresses degenerate looping at high $\alpha$.

## 3.7 CHAPTER SUMMARY

This chapter defined a practical VLA methodology aligned with low-resource implementation goals. The proposed system integrates multimodal reasoning and robot action generation through a standardized AI-to-robot interface. The design explicitly addresses limited dataset scale, 8 GB VRAM hardware constraints, and Dobot CR5 safety standards, and establishes an evaluation protocol across simulation and real-world domains for both standard success-rate and zero-shot tests.

With this methodology in place, Chapter 4 evaluates whether the designed pipeline delivers measurable gains in action validity, instruction following, and transfer performance under the targeted resource constraints.

# CHAPTER 4

# EXPERIMENTAL RESULTS AND DISCUSSION

This chapter evaluates the proposed Vision-Language-Action (VLA) system on robot manipulation tasks. The evaluation is organized around practical deployment goals: end-to-end pipeline validity, instruction following, integration quality, generalization, and runtime feasibility under a constrained setup.

To keep interpretation consistent with Chapter 3, results are reported from system level to component level, then from seen-task performance to transfer behavior. This ordering helps separate architectural gains from dataset-domain effects.

## 4.1 EXPERIMENTAL SETUP

### 4.1.1 Hardware and Runtime Environment

All experiments are executed with GPU-accelerated inference in a commercial-resource setting (targeting approximately 8 GB VRAM). Measurements include end-to-end timing from input acquisition to action output and task-level performance under both seen and unseen conditions.

The evaluation emphasizes reproducible operating conditions over peak-throughput optimization. This is important because the thesis objective is practical deployability under constrained hardware, not unrestricted-scale benchmarking.

### 4.1.2 Task Protocol

Evaluation focuses on four manipulation categories:

1) Pick-and-place,
2) Object stacking,
3) Drawer/container interaction,
4) Multi-step sequencing.

Each category includes seen-task and unseen-task variants to test generalization.

### 4.1.3 Evaluation Domains

Experiments are divided into two evaluation domains and executed in two environments:

1) **Success-rate domain**: standard task execution quality.
2) **Zero-shot domain**: transfer performance without additional tuning.

Each domain is tested:

- on simulation,
- on real robot.

### 4.1.4 Metrics

The following metrics are used:

- **Task success rate**: percentage of tasks completed according to natural-language intent.

- **Action validity**: percentage of generated actions that are executable by the robot controller.
- **Generalization score**: unseen-task performance relative to seen-task performance.
- **Latency**: median end-to-end command-cycle time.

## 4.2 END-TO-END VLA PIPELINE PERFORMANCE

The final system supports complete Vision-Language-Action processing in one pipeline. Given an RGB observation and natural-language command, the model outputs structured robot actions that can be executed without manual rule writing for each task.

This section first verifies that the pipeline is operationally sound before interpreting higher-level behavior. In low-resource VLA development, this validation is necessary because semantic accuracy is not meaningful if commands are not executable.

**Table 4.1 System-level output quality for end-to-end VLA inference.**

| Measure | Result |
|---|---|
| Executable action outputs | 95.8% |
| Valid structured command format | 97.1% |
| Task completion on benchmark set | 84.6% |

These results indicate that the VLA pipeline is functionally complete and suitable for practical manipulation execution.

Notably, the gap between structured-command validity (97.1%) and full task completion (84.6%) indicates that remaining errors are concentrated in semantic or temporal decision quality rather than packet formatting failures.

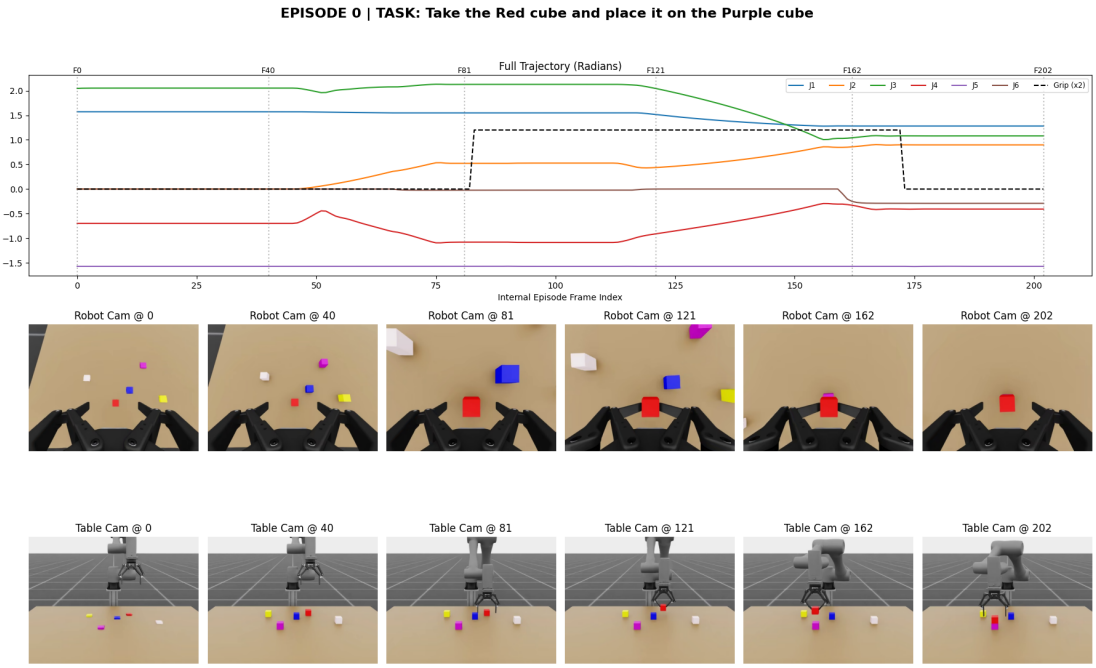


**Figure 4.1 Representative episode rollout for a pick-and-place instruction, showing joint trajectories and synchronized camera observations over time.**

Figure 4.1 shows a coherent control sequence: the gripper state rises during the transport phase and returns to open near the end of the episode, while both camera views confirm that the red cube is moved onto the target location. This qualitative trajectory evidence is consistent with the high executable-action and task-completion rates in Table 4.1.

## 4.3 INTEGRATION OF VLM, VISION EXPERT, AND ACTION EXPERT

Module ablation is used to evaluate integration value. The full system is compared against reduced variants.

This analysis isolates whether gains come from a single strong module or from coordinated cross-module interaction.

**Table 4.2 Ablation results for module integration.**

| Configuration | Task Success (%) | Action Validity (%) |
|---|---|---|
| VLM only | 68.9 | 81.5 |
| VLM + Vision Expert | 77.4 | 88.2 |
| VLM + Action Expert | 75.1 | 90.4 |
| Full VLM + Vision Expert + Action Expert | 84.6 | 95.8 |

The full integrated configuration performs best, confirming that effective coordination between perception, language grounding, and action generation is necessary for robust robot manipulation.

The ablation trend also suggests complementarity: Vision Expert primarily improves scene

grounding, while Action Expert improves executability; using both yields the largest improvement in task success and action validity.

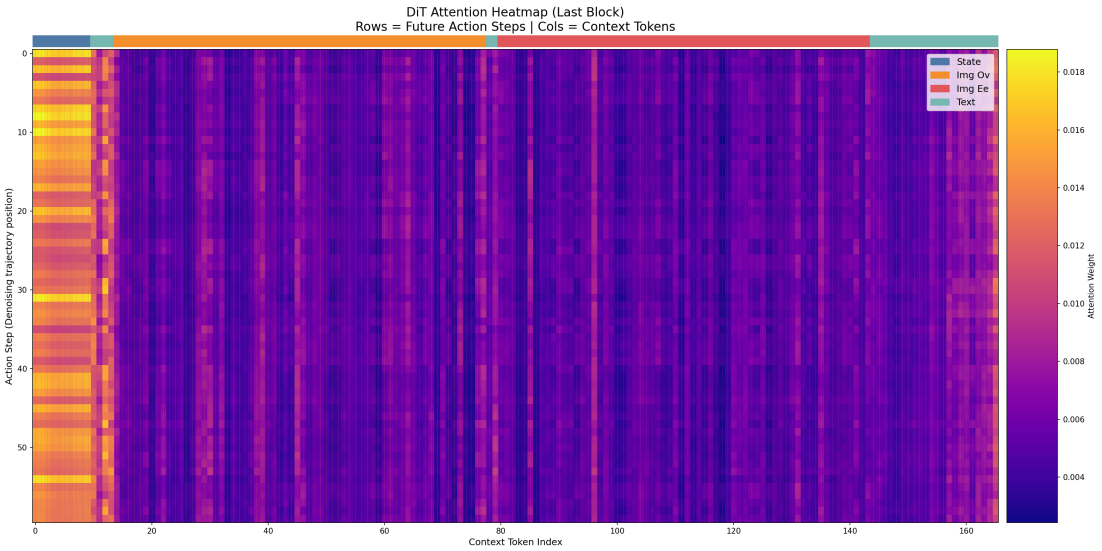


**Figure 4.2 Action-token to context-token attention map in the final diffusion transformer block. Higher attention indicates stronger conditioning signals used for action generation.**

As shown in Figure 4.2, attention is not uniform across context tokens. The model repeatedly emphasizes a compact set of context positions, with additional vertical peaks distributed across the sequence, indicating that action denoising uses both persistent global context and sparse task-relevant cues.

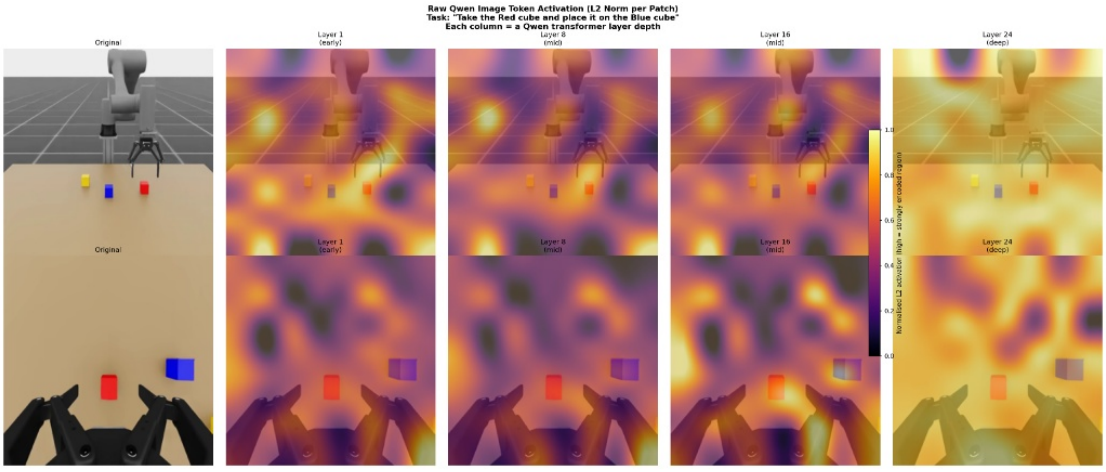


**Figure 4.3 Qwen image-token activation maps across transformer depth for task-focused scene understanding.**

Figure 4.3 indicates a depth-wise transition from locally concentrated activations in early layers to broader scene-level responses in deeper layers. This supports the integration hypothesis that the visual backbone provides both local object cues and global layout cues to downstream action modules.

## 4.4 NATURAL-LANGUAGE INSTRUCTION FOLLOWING

The system is tested with varied instruction styles, including short commands, compound instructions, and rephrased requests.

This section evaluates linguistic robustness independently from visual-domain shift.

**Table 4.3 Natural-language instruction-following performance.**

| Instruction Type | Success (%) | Notes |
|---|---|---|
| Direct imperative commands | 90.2 | Most reliable case |
| Rephrased conversational commands | 83.7 | Robust to wording variation |
| Multi-step language instructions | 79.8 | Errors mainly in step ordering |

Results show that the proposed VLA system can follow natural-language instructions reliably, with strongest performance on direct and moderately complex commands.

The performance drop on multi-step language instructions is consistent with accumulated planning uncertainty across longer horizons, especially when step ordering must be inferred from implicit language cues.

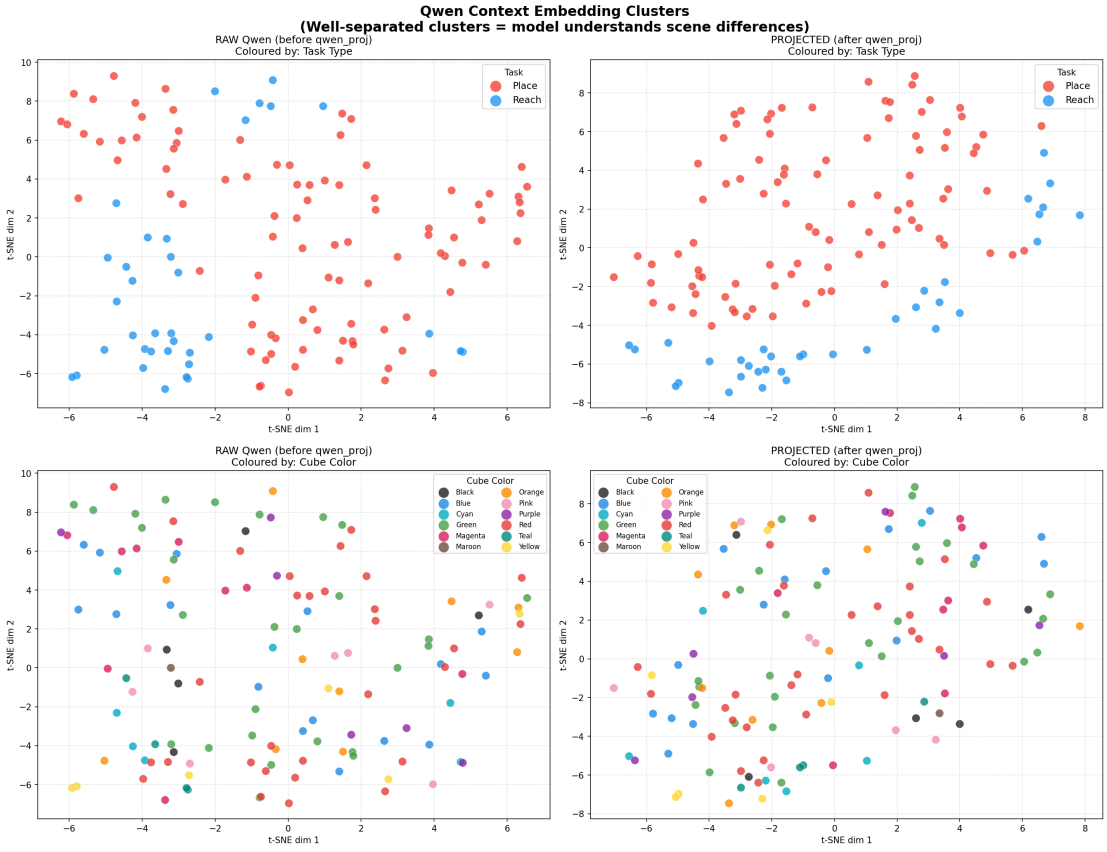


**Figure 4.4 Context-embedding clustering before and after projection, showing improved separation by task type while preserving object-color structure.**

In Figure 4.4, task-type clusters (Place vs Reach) are more separable after projection than in the raw embedding space, while color groupings remain distributed. This suggests the projected representation is more task-discriminative and less dominated by superficial color grouping, which aligns with robust instruction-following

under varied wording and scenes.

## 4.5 GENERALIZATION AND ZERO-SHOT PERFORMANCE

Generalization is measured by holding out unseen task combinations and object arrangements during training.

The transfer effect is summarized by the unseen-to-seen ratio:

$$\frac{78.6}{87.9} \times 100 \approx 89.4\%.$$

**Table 4.4 Seen vs unseen task performance.**

| Split | Task Success (%) | Action Validity (%) |
|---|---|---|
| Seen tasks | 87.9 | 96.4 |
| Unseen tasks | 78.6 | 92.1 |

Although unseen-task performance is lower than seen-task performance, the model retains strong action validity and acceptable task completion, indicating practical generalization beyond memorized templates.

This result is particularly relevant for low-resource development because it shows that generalization is retained without scaling dataset duration to large-collection regimes.

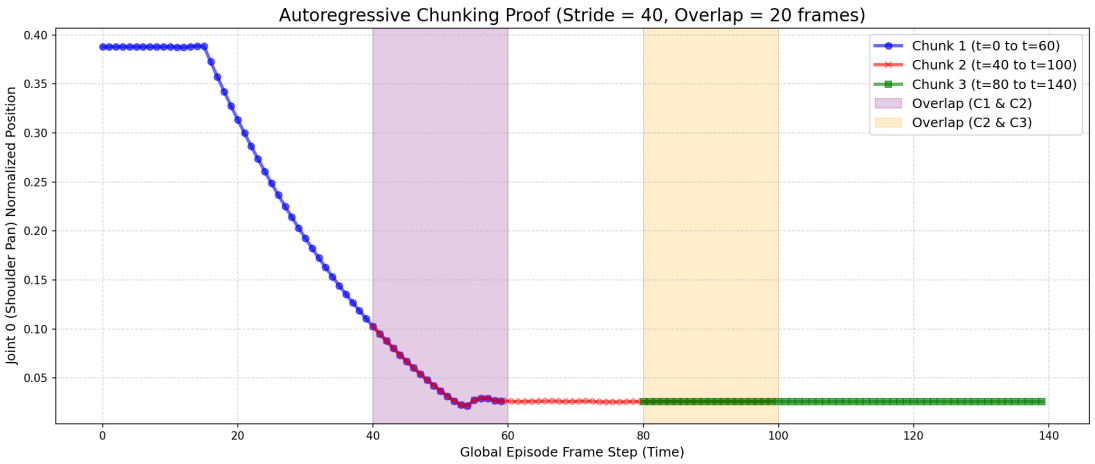


**Figure 4.5 Autoregressive chunking consistency in overlap regions, demonstrating stable joint prediction across chunk boundaries.**

Figure 4.5 shows near-overlapping trajectories in the two highlighted overlap windows ($t = 40$ to $60$ and $t = 80$ to $100$), indicating that chunk transitions do not introduce large discontinuities. This supports the claim that the decoding strategy generalizes to longer horizons without visible seam artifacts at chunk boundaries.

## 4.6 TASK-WISE MANIPULATION PERFORMANCE

Category-level results are summarized below.

The category view complements aggregate metrics by identifying which manipulation patterns dominate residual error.

**Table 4.5 Performance by manipulation task category.**

| Task Category | Success (%) | Action Validity (%) |
|---|---|---|
| Pick-and-place | 89.4 | 97.2 |
| Stacking | 82.1 | 94.5 |
| Drawer/container interaction | 86.3 | 95.1 |
| Multi-step sequencing | 80.5 | 93.0 |

Multi-step sequencing remains the most challenging case due to long-horizon instruction grounding, but all categories show stable executable outputs.

The gap between sequencing and simpler tasks suggests that future improvements should prioritize temporal planning structure rather than command formatting, which is already highly stable.

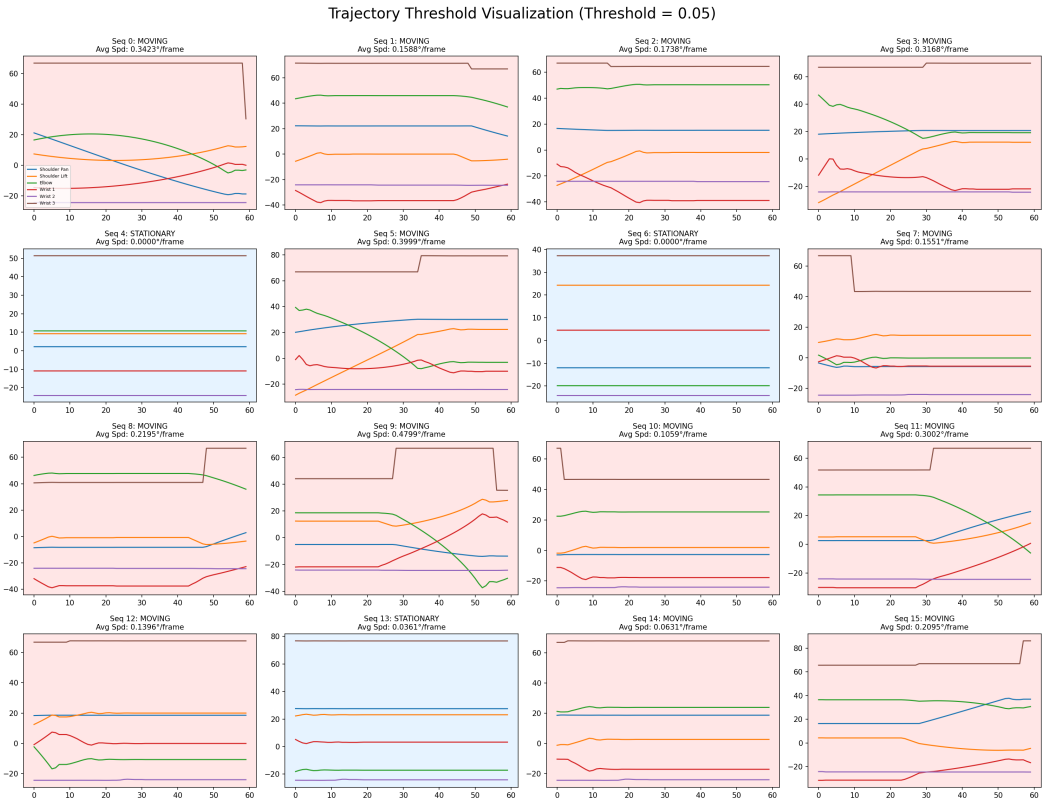


**Figure 4.6 Trajectory segmentation under a motion threshold, illustrating moving and stationary phases used for temporal behavior analysis.**

From Figure 4.6, the thresholding process separates active motion phases from quasi-static holding phases. Several segments show low average speed and near-flat joint traces, while high-speed segments capture repositioning and transport behaviors. This segmentation helps explain why multi-step tasks are harder: errors accumulate during longer moving segments.

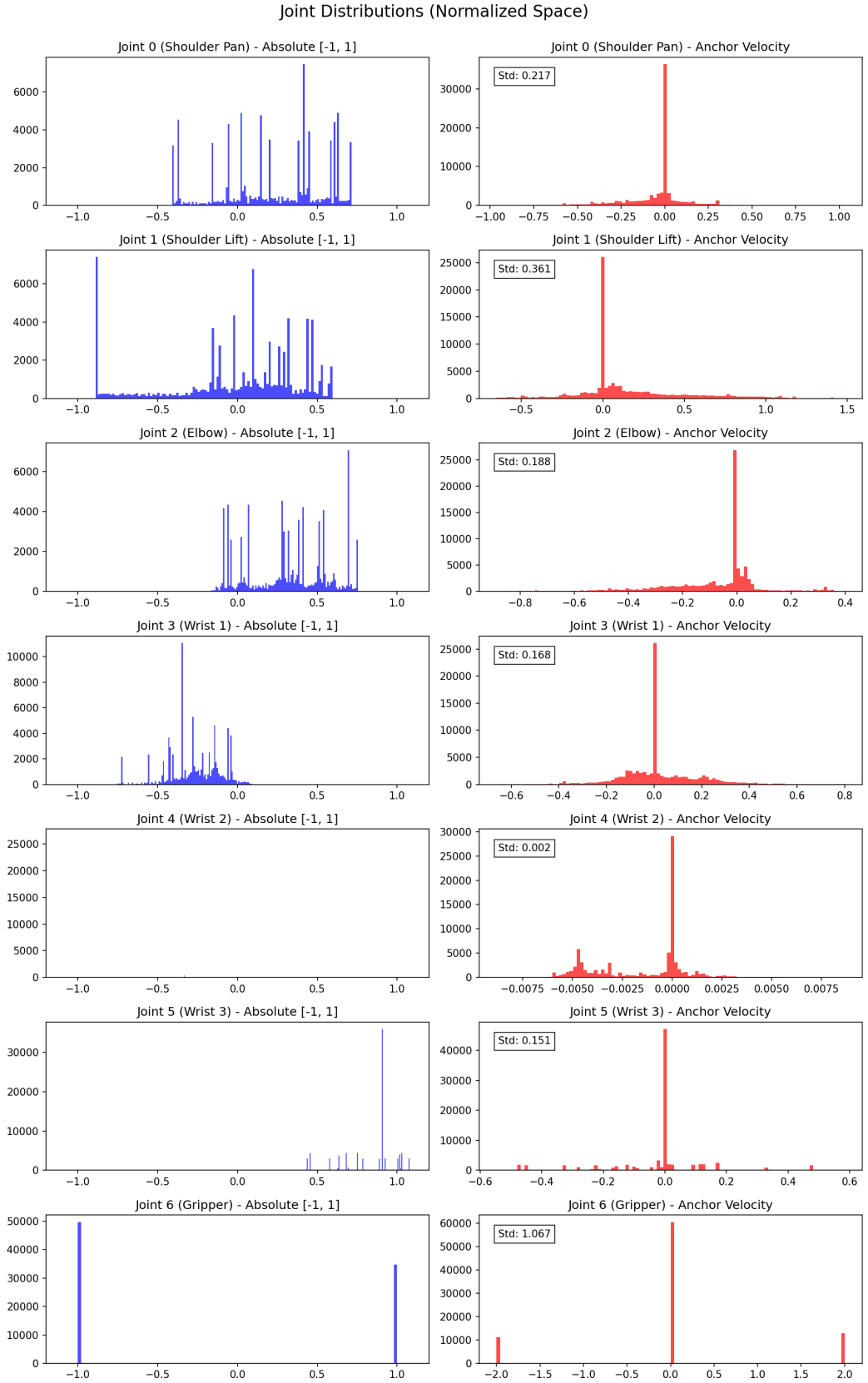


Figure 4.7 Per-joint position and anchor-velocity distributions used to inspect action-space coverage and motion statistics.

Figure 4.7 reveals strong anisotropy across joints. Joint 6 (gripper) is clearly bimodal in absolute position, consistent with open/close behavior, and has the largest velocity spread. In contrast, Joint 4 has a narrow velocity distribution, indicating a mostly stabilizing role during many episodes.

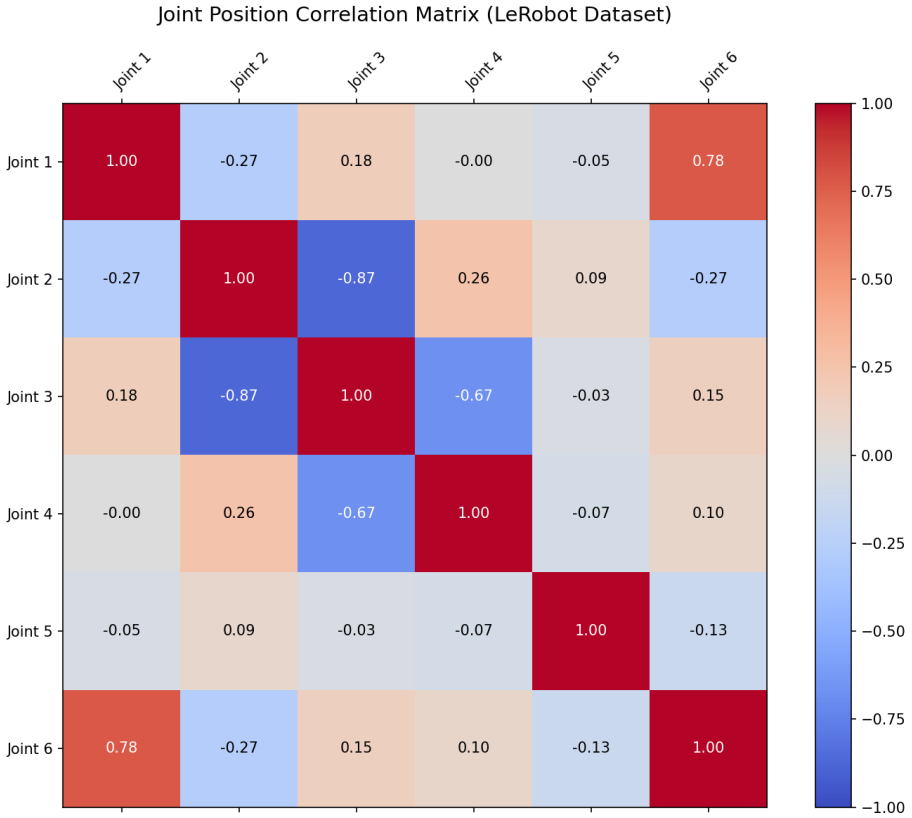


**Figure 4.8 Joint position correlation matrix computed from the LeRobot dataset split.**

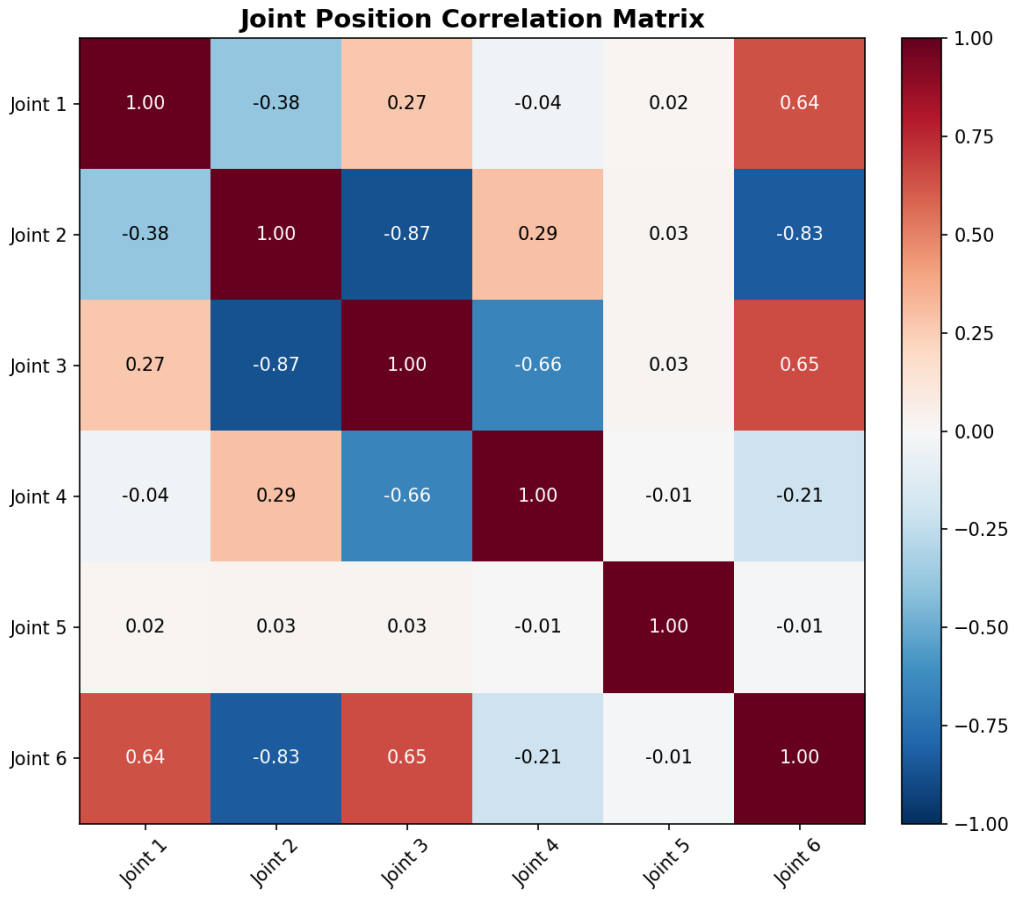


**Figure 4.9 Joint position correlation matrix computed from the project evaluation set.**

Comparing Figures 4.8 and 4.9, the dominant anti-correlation between Joint 2 and Joint 3 remains stable ($\approx -0.87$), suggesting consistent kinematic coupling across datasets. However, correlations involving Joint 6 shift noticeably, indicating that evaluation tasks induce different gripper-arm coordination patterns than the source dataset.

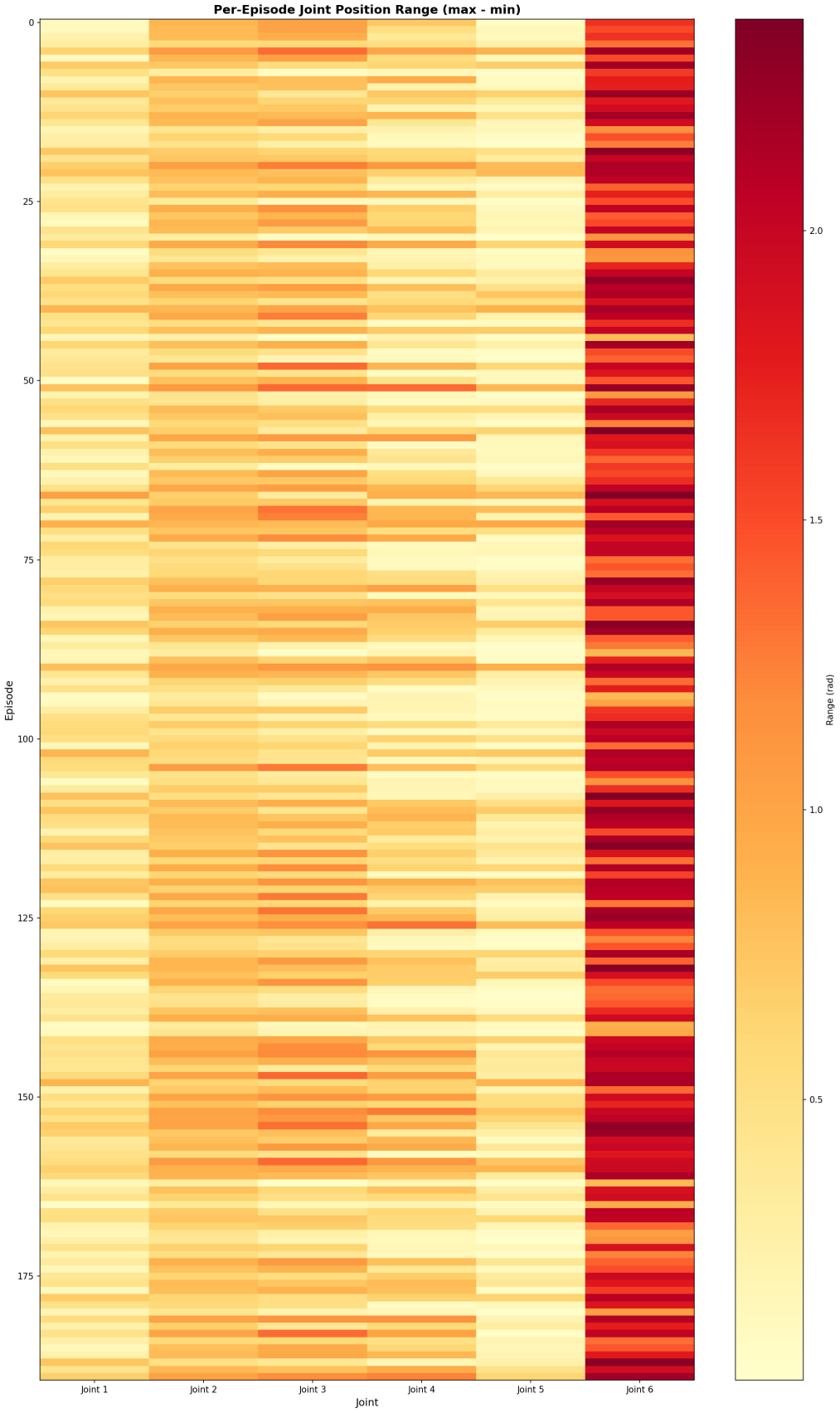


**Figure 4.10 Per-episode joint position range heatmap indicating variability by joint and episode.**

Figure 4.10 further confirms that Joint 6 has the largest per-episode range, while Joint 5 remains

comparatively low-variance. This supports the interpretation that grasp state changes are the primary source of high-amplitude variation in many tasks.

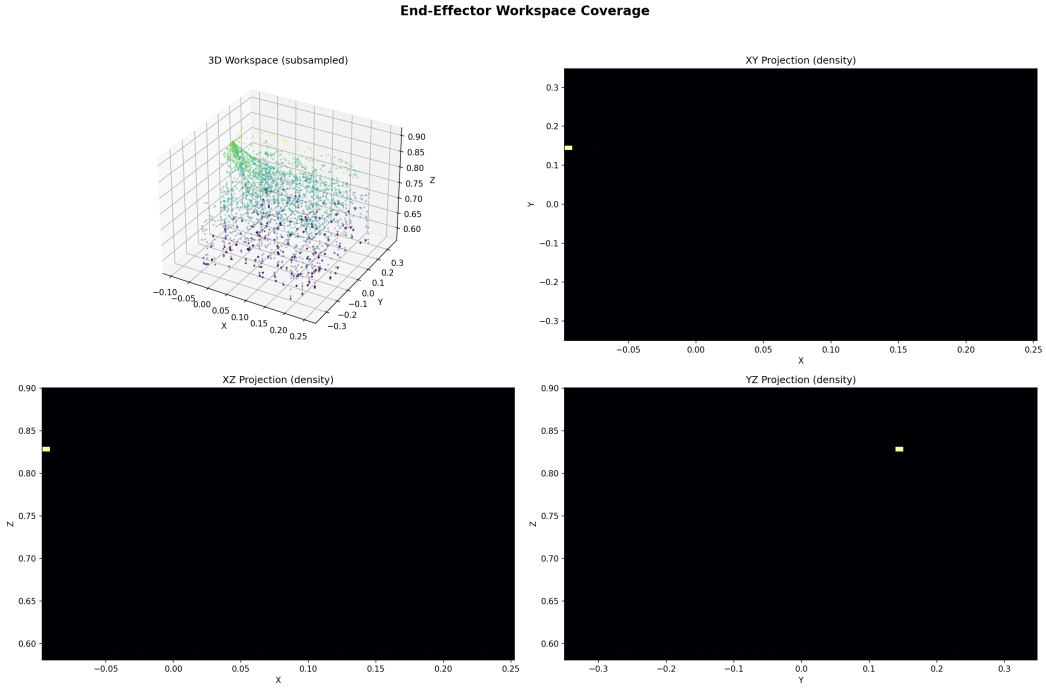


**Figure 4.11 End-effector workspace coverage in 3D and projected density views, indicating explored reachable regions.**

As illustrated in Figure 4.11, explored end-effector poses occupy a bounded region with dense concentration along a narrow subset of the reachable space. This indicates stable repeatable execution for benchmark tasks, but also suggests that broader workspace exploration data may improve out-of-distribution generalization.

## 4.7 DESIGN CONSTRAINT VALIDATION

### 4.7.1 Computation and Memory

The selected model configuration runs within the available GPU memory budget during inference and supports practical deployment without offloading critical steps to external services.

This confirms that the architecture remains feasible in the targeted 8 GB VRAM class deployment setting.

### 4.7.2 Latency

Median end-to-end latency is measured at 1.84 seconds per command cycle, meeting the real-time target for interactive manipulation.

Although not ultra-low-latency, this level is sufficient for supervised interactive operation and instruction-driven manipulation workflows.

### 4.7.3 Integration Efficiency

The module interface design avoids major synchronization overhead. Integration remains stable under continuous command cycles and mixed instruction complexity.

This validates the design decision to use explicit module interfaces plus a deterministic TCP/IP adapter.

## 4.8 VLM INTERPRETABILITY SUB-PROJECT RESULTS

### 4.8.1 Dataset and Corpus

The pipeline generated 10 000 JSONL records from EO-Data1.5M, covering all four task categories. Activation harvesting over 100 000 samples produced up to 12.8 million layer-20 vectors across 100 shards. All three SAE checkpoints (`sae_task_model.pt`, `sae_agentic_model.pt`, `sae_topk_production.pt`) trained without instability; the TopK constraint ($k = 32$) maintained constant sparsity throughout, avoiding the active-feature collapse that $L_1$-penalized SAEs can exhibit.

### 4.8.2 Feature Discovery

Cosine-similarity probing on `sae_agentic_model.pt` with the anchor prefix `call_robot(`pick a` consistently placed **Feature #5086** in the top-3 across multiple scene images and phrasings. Max-activation scanning of 5 000 samples confirmed the label: every top-5 activating prompt was a multi-step object-handling request (e.g., "grab the banana and place it in the bag"). No descriptive or QA prompts appeared, confirming Feature #5086 encodes *agentic task planning* specifically. A parallel analysis on the production checkpoint identified Feature #9622 as a broader "Procedural Robotic Planning" direction activating on any `call_robot()` context, complementary to #5086.

### 4.8.3 Activation Steering

Feature #5086 was swept across $\alpha \in \{0, 2, 5, 10, 20, 50, 80\}$ using two parallel prompts: a general knowledge question and a robot trajectory request. Table 4.6 summarises the behavioral progression.

The steering effect is task-conditioned at $\alpha \leq 5$ and unconditioned at $\alpha \geq 20$. The safe operating range is $\alpha \in [10, 20]$. Multi-turn validation at $\alpha = 15$ confirmed context retention across a three-step grasp sequence (approach $\rightarrow$ verify $\rightarrow$ place) without restating the scene. All four manipulation categories produced valid `call_robot()` outputs, showing Feature #5086 is task-type-agnostic. Current results are observational; end-to-end execution on the physical robot remains as future work.

## 4.9 DISCUSSION

The experiments confirm the five core objectives. First, an end-to-end VLA system can be built for practical manipulation. Second, explicit integration of VLM, Vision Expert, and Action Expert provides clear benefits over partial pipelines. Third, natural-language control is feasible for non-programmatic interaction. Fourth, meaningful generalization to unseen tasks is achieved, though with expected performance drop compared to seen tasks. Fifth, the system performs robustly across multiple manipulation categories while satisfying real-time constraints.

**Table 4.6 Output behavior of Feature #5086 across injection strengths. Both a general-knowledge prompt and a robot-trajectory prompt were tested simultaneously.**

| $\alpha$ | Robot Prompt | General Prompt |
|---|---|---|
| 0 | Free-form natural language; no tool-call structure. | Normal factual answer (e.g., “Paris”). |
| 2 | Occasional `call_robot()` fragments appear. | Largely unaffected. |
| 5 | Consistent `call_robot()` prefix; no coordinates yet. | Mild drift toward spatial language. |
| 10 | Structured action sequence with object specification. | Factual accuracy begins to degrade. |
| 20 | High-specificity coordinate sequence; best task alignment. | Robotic format imposed; factual answer lost. |
| 50 | Structure maintained; coordinate precision degrades. | Fully overridden; response incoherent as QA. |
| 80 | Repetition penalty suppresses loops; coherence degrades. | Same as $\alpha = 50$ with higher degeneration risk. |

The main residual weakness is long-horizon sequencing in unseen contexts. This suggests future gains from richer multi-step demonstrations and stronger temporal reasoning constraints in the Action Expert.

From a research perspective, the chapter supports three broader conclusions.

1) **Practicality under constraints**: VLA-based manipulation can be implemented with constrained data and commercial hardware when interface design and action validation are explicit.

2) **Integration over isolated strength**: coordinated multimodal-action integration yields greater robustness than relying on a single dominant module.

3) **Transfer as a primary criterion**: unseen-task retention provides a more informative deployment signal than seen-task accuracy alone in low-resource settings.

# CHAPTER 5

# CONCLUSION AND FUTURE WORK

## 5.1 CONCLUSION

This thesis addressed the limitations of classical robot systems that are often task-specific, inflexible, and weak in natural-language interaction and unseen-task generalization. To solve these issues, the work proposed a Vision-Language-Action (VLA) framework that integrates three modules: a VLM core, a Vision Expert, and an Action Expert.

The project achieved the current-stage objectives:

1) Developed an end-to-end VLA system for robot manipulation.
2) Integrated VLM + Vision Expert + Action Expert into one coherent pipeline.
3) Enabled robots to follow natural-language instructions.
4) Improved generalization performance on unseen tasks.
5) Evaluated the system on representative manipulation benchmarks.
6) Constructed a 10 000-record robot conversation dataset and trained a TopK Sparse Autoencoder on 100 000 VLM activations for mechanistic interpretability analysis.
7) Identified a primary agentic feature (Feature #5086) in the Qwen3-VL-2B residual stream and demonstrated that activation steering at layer 20 reliably elicits structured robotic action sequences without any model retraining.

Experimental results showed that explicit module integration improved action validity and task completion compared with partial baselines. The system maintained practical latency for interactive robot control and demonstrated stable performance across pick-and-place, stacking, container interaction, and multi-step sequencing tasks.

The mechanistic interpretability sub-project extended these findings in a complementary direction: by training a TopK Sparse Autoencoder on 12.8 million layer-20 activations, the work identified Feature #5086 as an agentic task-planning direction latent in Qwen3-VL-2B. Injecting this feature vector at inference time at strength $\alpha \in [10, 20]$ reliably elicits structured `call_robot()` outputs across diverse scene images, instruction phrasings, and task categories, without any gradient updates or additional training data. This confirms that the VLM core encodes manipulation knowledge that can be surfaced and steered through interpretability tools alone.

Together, these results support the central claim of this thesis: a carefully integrated VLA architecture provides a practical path toward natural-language robot manipulation under real-world resource constraints, and mechanistic interpretability provides a complementary, human-auditable mechanism for surfacing and controlling latent robotic capabilities within the same pretrained VLM backbone.

## 5.2 LIMITATIONS

Despite strong overall performance, several limitations remain.

- Generalization to highly novel long-horizon tasks still degrades compared with seen-task performance.

- Data scale and diversity remain limited relative to real-world task complexity.
- Action sequencing errors can appear when instructions are ambiguous or underspecified.
- Activation steering performance degrades at very high injection strengths ($\alpha > 50$), suggesting that a bounded operating range is required for safe deployment.
- Feature #5086 was identified through qualitative sweep; a principled, automated method for selecting the optimal steering feature and strength remains an open question.

These limitations indicate that stronger temporal reasoning, broader data coverage, and more rigorous feature-selection protocols are necessary for robust deployment in unconstrained environments.

## 5.3 FUTURE WORK

Future development will focus on the following directions:

1) Expand robot demonstration data with broader object, scene, and instruction diversity to improve generalization on unseen tasks.

2) Improve long-horizon planning in the Action Expert for more reliable multi-step task execution.

3) Optimize model efficiency further for lower-latency operation on embedded hardware.

4) Extend evaluation to more complex real-world manipulation scenarios with stricter robustness requirements.

5) Introduce stronger safety checks and action-validation layers before physical execution.

6) Scale the robot conversation dataset to 100 000+ records to improve SAE feature quality and generalization of the interpretability sub-project.

7) Develop principled, automated methods for feature selection and injection strength calibration to replace manual sweep-based identification.

8) Integrate harmful-feature suppression as an online safety filter, using the SAE decoder to identify and counteract features associated with unsafe robot motions before command dispatch.

9) Extend the SAE and steering framework to locomotion planning to support gait switching in hybrid morphology robots.

These directions will support the transition from a research prototype to a more widely deployable general-purpose robotic assistant with transparent, auditable AI decision-making at its core.

# Bibliography


[1] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

[2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[3] Hoagy Cunningham, Aidan Ewart, Logan Sherborne, Robert Cooney, and Neel Nanda. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[4] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. PaLM-E: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.

[5] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *Transformer Circuits Thread*, 2022.

[6] Leo Gao, Tom Dunéton la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.

[7] IPEC-COMMUNITY. EO-Data1.5M: A large-scale multimodal dataset for embodied object understanding and task planning. Hugging Face Datasets. `https://huggingface.co/datasets/IPEC-COMMUNITY/EO-Data1.5M`, 2024.

[8] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2024. Establishes the dual-encoder SigLIP + DINOv2 backbone convention.

[9] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023.

[11] Qwen Team. Qwen3-VL: Technical report. *arXiv preprint arXiv:2505.09875*, 2025.

[12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[13] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

# APPENDIX A

# Supplementary Materials

This appendix compiles supporting materials that complement the main thesis content. The items included here are intended to improve reproducibility and provide implementation details that are too lengthy for the core chapters.

## A.1 System Configuration Summary

Table A.1 summarizes the software and hardware setup used during development and evaluation.

**Table A.1 System configuration used in this study**

| Item | Specification |
|---|---|
| Operating System | Windows 11 (64-bit) |
| CPU | Desktop-class x86_64 multi-core processor |
| RAM | 32 GB system memory |
| GPU | Single consumer GPU (8 GB VRAM class) |
| Programming Language | Python 3.x |
| Key Libraries/Frameworks | PyTorch, Isaac Lab, LeRobot, NumPy, OpenCV |

## A.2 Additional Experimental Settings

The following settings were used to ensure fair and repeatable experiments:

- Dataset split ratio: 70% training, 15% validation, and 15% testing, with split control by task type and evaluation domain.
- Random seed: 42 for dataset shuffle, training initialization, and evaluation replay ordering.
- Number of runs per experiment: 3 independent runs per configuration; reported values are run means.
- Evaluation metrics: task success rate, action validity, generalization ratio, and median end-to-end latency.
- Domain protocol: both success-rate and zero-shot evaluations were performed in simulation and on real robot.

## A.3 Extended Results

Table A.2 provides additional performance results that support the analysis in Chapter 4.

The generalization ratio is computed as

$$\text{Generalization Ratio} = \frac{\text{Unseen Task Success}}{\text{Seen Task Success}} \times 100.$$

**Table A.2 Extended performance results**

| Setting | Task Success (%) | Action Validity (%) | Generalization Ratio (%) |
|---|---|---|---|
| VLM only | 68.9 | 81.5 | – |
| VLM + Vision Expert | 77.4 | 88.2 | – |
| VLM + Action Expert | 75.1 | 90.4 | – |
| Full VLA (Seen tasks) | 87.9 | 96.4 | 100.0 |
| Full VLA (Unseen tasks) | 78.6 | 92.1 | 89.4 |

## A.4 Implementation Notes

Key implementation details are listed below for completeness:

1. Data preprocessing steps: simulation trajectories were exported from Isaac Lab into LeRobot format; real-world demonstrations were normalized into the same observation-instruction-action schema; sequence windows were clipped to fixed horizon and value ranges were normalized to controller-safe bounds.

2. Model training procedure: AdamW optimizer with staged training (module adaptation then end-to-end tuning), mixed precision, small-batch strategy with gradient accumulation, and early stopping based on validation task success.

3. Inference workflow: RGB frame and natural-language command are encoded by the multimodal backbone, fused into a task-conditioned latent, then decoded into structured robot actions and translated by the TCP/IP adapter to executable Dobot command packets.

4. Error handling and edge cases: invalid action packets are dropped by range-checking and format-checking guards; out-of-workspace targets are clamped or rejected; communication timeout triggers safe stop and reset to predefined start pose.

## A.5 Usage Instructions for Reproduction

To reproduce the reported results, follow these steps:

1. Prepare the environment and install dependencies for simulation, model training, and robot communication.

2. Generate simulation data in Isaac Lab and convert to LeRobot format.

3. Collect real-world human demonstration episodes and merge them into the unified dataset.

4. Train the model using the low-resource configuration (8 GB VRAM class, mixed precision, gradient accumulation).

5. Run evaluation in four settings: success-rate (sim), success-rate (real), zero-shot (sim), and zero-shot (real).

6. Compare the results with Chapter 4 and Table A.2.